Code for 4th:

```
import os, math
import numpy as np
from PIL import Image, ExifTags
import matplotlib.pyplot as plt

filenames = [
   "C:/Users/dhivy/OneDrive/dark.jpg",
   "C:/Users/dhivy/OneDrive/normal.jpg",
   "C:/Users/dhivy/OneDrive/bright.jpg"
]

out_dir = "hdr_outputs_nocv2"
os.makedirs(out_dir, exist_ok=True)

def load_image_rgb(path):
   img = Image.open(path).convert("RGB")
   return np.asarray(img, dtype=np.float32) / 255.0

imgs = [load_image_rgb(fn) for fn in filenames]

def get_exposure_times_from_exif(filenames):
   times = []
   for fn in filenames:
      try:
         img = Image.open(fn)
         exif = img._getexif()
         if not exif:
            times.append(None)
            continue
         val = None
         for tag_id, v in exif.items():
            tag = ExifTags.TAGS.get(tag_id, tag_id)
            if tag == "ExposureTime":
               if isinstance(v, tuple):
                  val = float(v[0]) / float(v[1]) if v[1] != 0 else None
               else:
                  val = float(v)
               break
         times.append(val)
      except Exception:
         times.append(None)
   if any(t is None for t in times):
      return None
   return np.array(times, dtype=np.float32)

exif_times = get_exposure_times_from_exif(filenames)
```

```
if exif_times is not None:
    times = exif_times
else:
    times = np.array([1/1000.0, 1/125.0, 1/15.0], dtype=np.float32)

times = times.reshape(-1, 1, 1, 1)

imgs_stack = np.stack(imgs, axis=0)

eps = 1e-6
gray_stack = 0.299*imgs_stack[...,0] + 0.587*imgs_stack[...,1] + 0.114*imgs_stack[...,2]
w_well = np.exp(-4.0 * (gray_stack - 0.5)**2) + eps
w_well = w_well[..., None]

num = np.sum(w_well * imgs_stack / times, axis=0)
den = np.sum(w_well / times, axis=0)
hdr = num / (den + eps)

np.save(os.path.join(out_dir, "hdr_radiance.npy"), hdr)

def tonemap_reinhard(hdr, gamma=2.2, key=0.18):
    eps = 1e-6
    R, G, B = hdr[...,0], hdr[...,1], hdr[...,2]
    L = 0.2126*R + 0.7152*G + 0.0722*B
    log_L = np.log(L + eps)
    L_avg = np.exp(np.mean(log_L))
    Lm = (key / (L_avg + eps)) * L
    Ld = Lm / (1.0 + Lm)
    scale = Ld / (L + eps)
    out = hdr * scale[..., None]
    out = np.clip(out, 0.0, 1.0)
    out = out ** (1.0/gamma)
    return out

ldr = tonemap_reinhard(hdr, gamma=2.2)
ldr_8 = (np.clip(ldr,0,1) * 255).astype(np.uint8)
Image.fromarray(ldr_8).save(os.path.join(out_dir, "tonemapped_reinhard.png"))

def exposure_fusion_like(imgs_stack):
    eps = 1e-12
    gray = 0.299*imgs_stack[...,0] + 0.587*imgs_stack[...,1] + 0.114*imgs_stack[...,2]
    w_well = np.exp(-4.0 * (gray - 0.5)**2)
    sat = np.std(imgs_stack, axis=3)
    weights = w_well * (sat + eps)
    w_sum = np.sum(weights, axis=0, keepdims=True) + eps
    W = weights / w_sum
    fused = np.sum(W[...,None] * imgs_stack, axis=0)
    return fused
```

```
fused = exposure_fusion_like(imgs_stack)
fused_8 = (np.clip(fused,0,1) * 255).astype(np.uint8)
Image.fromarray(fused_8).save(os.path.join(out_dir, "exposure_fused.png"))

# -------- overall comparison figure --------
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
axes[0].imshow(imgs[1])
axes[0].set_title("Normal")
axes[0].axis("off")
axes[1].imshow(ldr_8)
axes[1].set_title("Tone-mapped HDR")
axes[1].axis("off")
axes[2].imshow(fused_8)
axes[2].set_title("Exposure Fusion")
axes[2].axis("off")
plt.tight_layout()
plt.show()

# -------- tone-mapping curve figure --------
L_vals = np.linspace(0.001, 5.0, 500)
key = 0.18
L_avg_assumed = 1.0
Lm = (key / L_avg_assumed) * L_vals
Ld = Lm / (1.0 + Lm)
plt.figure(figsize=(6,4))
plt.plot(L_vals, Ld)
plt.xlabel("Input luminance L")
plt.ylabel("Output luminance Ld")
plt.title("Reinhard Tone-Mapping Curve")
plt.grid(True)
plt.tight_layout()
plt.show()

print("Done. Outputs in:", out_dir)
```